yuktai

Developer Learning & Integration Guide

Accessible, agentic AI for React, Next.js and plain HTML that runs in the browser

Package: `@yuktishaalaa/yuktai` · **Version covered:** 5.3.4 · **Licence:** GPL-3.0

Written for: web developers building for English-speaking users and for users of Telugu, Hindi and the other Indian languages.

How this guide was checked: every function, prop, phrase and sample answer in this guide was run against version 5.2.0 of the package, and the “What’s new in 5.3” section against 5.3.4 on 24 October 2026. Where a feature has a limit, the guide says so.

Contents

What's new in 5.3 (October 2026)
1. What yuktai is
2. Languages: what works today
3. Install and set up
4. YuktaiGrid basics
5. The agentic grid: useAgenticGrid + AgenticGridBar
6. The built-in grid assistant, agent tools and WebMCP
7. Page agents
8. All page tools in one panel: AgenticPanel
9. Core modules
10. Large datasets (100K+ rows)
11. Plug in your own model (optional)
12. Honest limits
13. Compatibility, principles and links

What's new in 5.3 (October 2026)

Version 5.3 brings yuktai to plain HTML pages and adds a set of 163 icons for Next.js, React and plain HTML, with scripts that move whole projects to them. Everything in sections 1–13 still works the same way.

Version	What changed
5.3.0	yuktai for plain HTML, CSS and JavaScript: every module from one script tag, built from the same source as the npm package. The live demo is an installable, offline app. Accessibility engine fixes.
5.3.1	41 icons for Next.js and React from a new entry, `@yuktishaalaa/yuktai/icons`, that also works in Server Components. Outline and filled styles with WCAG contrast checks.
5.3.2	33 more icons, 74 in total: home, menu, book, calendar, clock, lamp, temple, meditate, share, play, pause, sun and more.
5.3.3	58 more icons, 132 in total: delete, robot, heart, the text-editor set, lock, link, storage, cloud, Wi-Fi and more. Filled parts follow the glyph colour.
5.3.4	31 more icons, 163 in total: charts, zoom, save, login and logout, sunrise and sunset, namaskar, star, shield and more.

Icons in Next.js

Install, then import from the icons entry. It has no hooks and no browser code, so no `"use client"` is needed.

```
npm install @yuktishaalaa/yuktai@5.3.4
```

```
import { HomeIcon, MicIcon, SearchIcon, LampIcon, YuktaiIcon } from "@yuktishaalaa/yuktai/icons"

<button type="button" aria-label="Start voice input"><MicIcon /></button>
<SearchIcon size={24} label="Search" />
<LampIcon variant="filled" color="#005A9C" size={32} />
<YuktaiIcon name="temple" />
```

Prop	Default	What it does
`size`	20	Width and height in pixels
`color`	currentColor	Line colour; for filled icons, the background colour
`variant`	"outline"	"outline" or "filled"
`glyphColor`	automatic	Filled only: white or dark, whichever has more contrast
`shape`	"rounded"	Filled only: "rounded" square or "circle"
`strokeWidth`	2.5	Line thickness
`label`	none	Gives the icon a name for screen readers. Without it the icon is decorative and hidden from them.

Every icon name has a component: capital first letter plus Icon, so `home` is `HomeIcon` and `arrowUp` is `ArrowUpIcon` (`left` and `right` are `ChevronLeftIcon` and `ChevronRightIcon`). Any icon also works as `<YuktaiIcon name="..." />`, and `ICON_NAMES` lists all 163.

Icons in plain HTML

```
<span data-icon="home"></span>
<script src="https://cdn.jsdelivr.net/npm/@yuktishaalaa/yuktai@5/vanilla/index.min.js"></script>
```

```
<script>
  document.querySelectorAll("[data-icon]").forEach(function (el) {
    el.innerHTML = yuktai.icon(el.dataset.icon, 22)
  })
</script>
```

Options work the same way: `yuktai.icon("temple", { size: 32, variant: "filled", color: "#005A9C", label: "Temple" })`.

All 163 icon names

From 5.3.1 (41): accessibility, contrast, moon, motion, reading, target, speaker, palette, skip, close, plus, minus, reset, search, sort, sortUp, sortDown, chevrons, grid, cards, chat, mic, stop, download, copy, filter, check, left, right, first, last, spark, alert, info, form, eye, layout, wand, undo, tools, plug.

New in 5.3.2 (33): home, menu, book, calendar, clock, person, users, globe, lamp, play, pause, send, share, gift, image, file, external, arrowUp, arrowLeft, arrowRight, headphones, music, pin, map, pen, wallet, install, help, temple, meditate, trophy, bulb, sun.

New in 5.3.3 (58): delete, robot, heart, heartFilled, hub, code, text, bold, italic, underline, alignLeft, alignCenter, alignRight, alignJustify, quote, title, font, spellcheck, abc, pageBreak, link, style, lock, unlock, key, redo, swap, shuffle, fullscreen, fullscreenExit, merge, join, touch, volumeOff, musicOff, musicVideo, movie, slideshow, storage, api, dataObject, computer, phone, cloud, wifi, wifiOff, searchOff, eyeOff, bookmark, school, gavel, architecture, brain, volunteer, radio, radioChecked, upload, arrowDown.

New in 5.3.4 (31): factCheck, imageSearch, documentScanner, dashboard, cleaning, dragHandle, zoomIn, zoomOut, insights, explore, save, label, keyboard, trendingUp, trendingDown, cake, barChart, admin, bugReport, sentimentSad, integration, namaskar, sunrise, sunset, login, logout, speed, repeat, loader, star, shieldCheck.

WCAG rules built into the icons

- **1.1.1 Non-text content:** icons are decorative by default. Pass `label` only when the icon means something on its own.
- **1.4.11 Non-text contrast:** filled icons pick a glyph colour with at least 3:1 contrast and warn in the console if a custom colour is lower. For outline icons, choose a colour with 3:1 against your background, for example W3C blue #005A9C on white (7.1:1).
- **Forced colours (Windows High Contrast):** outline icons follow the text colour; filled icons switch to system colours.
- **2.5.8 Target size:** an icon is not a target. Put it inside a button of at least 24×24 px, or 44×44 px on touch screens.

Plain HTML: the whole toolkit from one script tag

The vanilla build gives a page without React the same grid, page agents and accessibility engine. Add the script, then call the functions on `window.yuktai`.

```
<div id="staff"></div>
<script src="https://cdn.jsdelivr.net/npm/@yuktishaalaa/yuktai@5/vanilla/index.min.js"></script>
<script>
  const grid = yuktai.renderGrid("#staff", { data: rows, columns, rowKey: "id", agentic: true,
ai: true })
  yuktai.mountAgenticPanel("#tools", { defaultTab: "qa" })
</script>
```

Live demo with every feature: https://sandeepmiriyala03.github.io/yuktai/vanilla/

Switching a whole project to yuktai icons

Four small Node scripts move an existing project to yuktai icons. Keep them together in one folder. None of them needs anything installed except yuktai itself.

Script	What it does
`scan-icons.mjs`	Read-only. Counts the icons a project uses (Material, Lucide, Feather, Font Awesome, react-icons, icon fonts, emoji) and how many yuktai can replace.
`use-yuktai-icons.mjs`	Switches Material, Lucide, Feather and react-icons imports to yuktai. Material icons are drawn inside MUI's `SvgIcon`, so `sx`, `fontSize` and `color` keep working; only import lines change.
`review-yuktai-icons.mjs`	Builds `icons-review.html`: every original icon next to its yuktai replacement, with the files that use it, so meanings can be checked by eye.
`run-all-icons.mjs`	Runs the scan and the switch for every project in a folder. Reports first; with `--apply` it only changes committed git projects, so `git checkout -- .` undoes everything.

```
npm install @yuktishaalaa/yuktai@latest
node scan-icons.mjs                      # what is there, nothing changes
node use-yuktai-icons.mjs --dry          # preview
node use-yuktai-icons.mjs                # switch
node review-yuktai-icons.mjs             # check every pair
npx tsc --noEmit
node run-all-icons.mjs C:\path\to\projects --apply --install   # many projects at once
```

The switch script only uses icons the installed yuktai version has. With an older version it switches what it can and says how many more an update would switch.

Results in October 2026, after the switch:

Site	Before	After
Ratnalabala	393 Material + 7 Feather imports	100% yuktai icons
AksharaTantra (AIHACK)	110 Material imports	100% yuktai icons

Brand logos have no yuktai drawing; they switch to the action they stand for: WhatsApp → share, Telegram → send, Android → phone.

Accessibility engine fixes in 5.3

- Real labels on form fields and buttons are no longer replaced.
- Hidden or icon-only buttons get a correct name.
- A header or footer inside another section no longer becomes an extra page landmark.
- The fix count only counts fixes that were really applied.
- Alert and skip-link colours meet 4.5:1 contrast; information alerts use W3C blue #005A9C.

Honest limits found in October 2026 testing

We asked the demo grid 17 questions. Filters with numbers ("salary between 60000 and 90000", "50k to 80k"), counts, sorting and the unusual-value check worked. These did not, and are planned fixes:

- "salary over 1 lakh in Hyderabad" applied the salary filter but dropped the city.

- "not in Hyderabad" showed the Hyderabad rows, and "Chennai or Pune" showed only Chennai.
- "average salary by department" reported a single department group.
- "top 5 salaries", "who earns the most", "joined after 2020", "status inactive" and the Telugu question "హైదరాబాద్ లో ఎంత మంది" were not understood.

Until these are fixed, try the questions your users will really ask, and add `customRules` for the important ones (section 6).

Links

What	Where
GitHub repository	https://github.com/sandeepmiriyala03/yuktai
npm package	https://www.npmjs.com/package/@yuktishaalaa/yuktai
Live demo (plain HTML, installable)	https://sandeepmiriyala03.github.io/yuktai/vanilla/
Start page	https://sandeepmiriyala03.github.io/yuktai/
Grid demo, 10 lakh rows	https://aksharatantra.miriyala.in/staffdirectory
About yuktai page	https://aksharatantra.miriyala.in/about
CDN (jsDelivr)	https://cdn.jsdelivr.net/npm/@yuktishaalaa/yuktai@5/vanilla/index.min.js
CDN (unpkg)	https://unpkg.com/@yuktishaalaa/yuktai@5/vanilla/index.min.js
Issues and ideas	https://github.com/sandeepmiriyala03/yuktai/issues
Security policy	https://github.com/sandeepmiriyala03/yuktai/security/policy
XERJ contributor profile	https://hub.xerj.org/contributors/sandeepmiriyala03
XERJ contributor badge	https://hub.xerj.org/contributors/sandeepmiriyala03/badge.svg
XERJ yuktai corpus page	https://hub.xerj.org/corpus/sandeepmiriyala03--yuktai

1. What yuktai is

yuktai is one npm package that gives a React or Next.js app four things: an accessible **data grid you can talk to**, **agents** that test and explain a page, **WCAG accessibility** fixes, and **question answering** over the page and its data.

Everything runs in the browser with rule-based logic and keyword search. There are no API keys, no server calls and no cost. Your page and grid data stay on the device unless you plug in your own model on purpose.

Fact	Value
Latest version	5.3.4
Frameworks	React 17–19, Next.js 13–16 (App Router), plain HTML (vanilla build)
Runs where	In the browser; SSR-safe (no `window` errors on the server)
API keys / cost	None / free
Optional AI	Chrome's built-in Gemini Nano, Transformers.js, or your own model through a hook
Types	Full TypeScript definitions

The six modules

Module	What it gives you
1. Accessibility engine	WCAG 2.2 auto-fix, speak on focus, colour-blind modes, dyslexia font, skip links
2. YuktaiGrid	Accessible data grid with an assistant, agent tools, WebMCP and 5 WCAG themes
3. In-tab RAG	Ask questions about the current page, offline
4. Autonomous agent	Plain-English goal → reads the page → highlights the steps
5. Vibe Coder	Describe a site → download a Next.js project (templates, no AI-written code)
6. Agentic AI toolkit (v5)	10 agentic features: grid questions, BI answers, tasks, anomalies, QA, copilot, forms, design, page Q&A, voice

The 10 agentic features at a glance

#	Feature	What it does	Main exports
1	Smart grid queries	"salary over 1 lakh" → real filters, sort, limit. Understands 10k, lakh, crore	`useAgenticGrid`, `parseGridQuery`, `runGridQuery`
2	BI copilot	Group by, this month vs last month, trends	`askInsight`, `groupBy`, `comparePeriods`, `trend`
3	Task runner	"filter, then sort, then summarize" as ordered steps	`planTasks`, `runTasks`
4	Anomaly finder	Unusual numbers and duplicate rows	`detectAnomalies`, `summarizeRows`
5	QA tester	15 WCAG checks, 0–100 score, Markdown/JSON report	`runQaAudit`, `auditToMarkdown`, `auditToJSON`
6	Accessibility copilot	Explains issues, applies safe fixes, undo	`createA11yCopilot`
7	Page understanding	"What is this page for?", "Which fields have no label?"	`askAboutPage`, `analyzePage`

#	Feature	What it does	Main exports
8	Form review	Labels, autocomplete, errors, senior-friendly tips	`reviewForms`, `reviewForm`
9	Design review	Hierarchy, readability, interaction, clutter, consistency	`reviewDesign`
10	Voice commands	"bigger text", "next row", Telugu phrases too	`parseVoiceA11yCommand`, `createVoiceA11yController`

Two React components bundle these for you: **AgenticGridBar** (a question bar for a grid) and **AgenticPanel** (one panel with six page-tool tabs).

2. Languages: what works today

yuktai is built for India as much as for English. This table separates what is fully supported from what is partial, so you can plan your UI honestly. Each row was tested on 5.2.0.

Area	English	Telugu	Hindi and other Indian languages
Grid UI text	Yes	Yes (`locale="te-IN"`)	Locale codes accepted for all 22 scheduled languages; built-in UI text is English/Telugu
Voice input to the grid assistant	Yes	Yes	Yes: `inputLanguage` accepts all 22 codes (uses the browser's speech recognition)
Built-in grid assistant phrases	count, search, open, sort, clear	ఎన్ని, ... వెతుకు, ... తెరువు, ... క్రమంలో	Not parsed yet; falls back to a search for the whole sentence
Smart grid queries (`useAgenticGrid`)	Yes	Not yet: numbers like "1 లక్ష" are misread	Not yet
Indian number words	10k, 1.5 lakh, 2cr, ₹10,000	—	—
Indian number format in answers	10,00,000 (en-IN)	Same	Same
Voice accessibility commands	Yes	Yes: పెద్ద అక్షరాలు, హై కాంట్రాస్ట్, డార్క్ మోడ్, మొదటి వరుస, రీసెట్...	Not yet (e.g. "बड़ा टेक्स्ट" is not recognised)
Keyword search (RAG index)	Yes	Yes, words kept whole	Yes for Devanagari and other scripts (tokenizer keeps combining marks)
Answers and reports	English	Grid tool messages in Telugu (`locale: "te"`); other answers in English	English

> **Tip for Indian-language apps:** use the grid's built-in assistant with `customRules` for your own Telugu (or Hindi) phrases. Rules match any script exactly, run before the built-in parser, and can call any grid tool. See section 6.4.

3. Install and set up

Install

```
# Next.js 16 + React 19 (recommended)
npm install @yuktishaalaa/yuktai --legacy-peer-deps

# Next.js 13, 14, 15
npm install @yuktishaalaa/yuktai

# Yarn / pnpm
yarn add @yuktishaalaa/yuktai
pnpm add @yuktishaalaa/yuktai
```

Requirements: Node.js 18+, npm 8+, Next.js 13+.

Step 1 — next.config.js

```
const nextConfig = {
  transpilePackages: ["@yuktishaalaa/yuktai"],
  async headers() {
    return [{
      source: "/(.*)",
      headers: [
        // Needed for voice control and voice input
        { key: "Permissions-Policy", value: "microphone=*, speaker=*" },
      ],
    }];
  },
};
module.exports = nextConfig;
```

Step 2 — client wrapper (floating tools on every page)

```
// components/YuktaiClient.tsx
"use client";
import { useState, useEffect, type ReactNode } from "react";
import { YuktAIWrapper } from "@yuktishaalaa/yuktai";

export default function YuktaiClient({ children }: { children: ReactNode }) {
  const [mounted, setMounted] = useState(false);
  useEffect(() => setMounted(true), []);
  if (!mounted) return <>{children}</>;
  return (
    <YuktAIWrapper position="left" showRag showAgent>
      {children}
    </YuktAIWrapper>
  );
}
```

Step 3 — app/layout.tsx

```
import YuktaiClient from "@/components/YuktaiClient";

export default function RootLayout({ children }: { children: React.ReactNode }) {
  return (
```

```
    <html lang="en">   {/* use lang="te", "hi", … for Indian-language sites */}
      <body>
        <YuktaiClient><main>{children}</main></YuktaiClient>
      </body>
    </html>
  );
}
```

Wrapper prop	Type	Default	Purpose
position	"left" | "right"	"left"	Side of the floating buttons
showRag	boolean	false	Show the 💬 Ask-this-page button
showAgent	boolean	false	Show the AI agent button
config	Partial<A11yConfig>	—	Accessibility overrides
children	ReactNode	required	Your app

> Set the right `lang` on `<html>`. The QA tester flags a missing language as **serious**, because screen readers pick their voice from it: without it, Telugu or Hindi text may be read with English rules.

4. YuktaiGrid basics

```
"use client";
import { YuktaiGrid } from "@yuktishaalaa/yuktai";

const rows = [
  { id: 1, name: "Ravi Kumar",  city: "Hyderabad", department: "Engineering", salary: 145000,
joined: "2024-03-12", status: "Active" },
  { id: 2, name: "Sneha Varma", city: "Bengaluru", department: "Design",      salary: 92000,
joined: "2025-08-03", status: "Active" },
];
const inr = (n: number) => "₹" + n.toLocaleString("en-IN");

export default function StaffPage() {
  return (
    <YuktaiGrid
      data={rows}
      rowKey="id"
      columns={[
        { key: "name",   label: "Name",   sortable: true },
        { key: "city",   label: "City",   sortable: true },
        { key: "salary", label: "Salary", type: "number", align: "right", render: (v) =>
inr(Number(v)) },
        { key: "joined", label: "Joined", type: "date" },
        { key: "status", label: "Status", type: "badge" },
      ]}
      locale="en-IN"          // or "te-IN", "hi-IN", …
      theme="default"         // default | high-contrast | dark | color-blind | dyslexia
      view="auto"             // table | card | auto (cards on small screens)
      search
      pagination={{ pageSize: 20, showSizeChanger: true, sizeOptions: [10, 20, 50, 100] }}
    />
  );
}
```

Grid features: real-time search, column sorting, pagination with page-size switching, mobile card view, loading and empty states, 5 WCAG themes, row selection, custom cell render, row highlighting with auto-scroll (`highlightIds`, `autoScrollToHighlight`), keyboard navigation and ARIA grid semantics with 44×44 targets.

5. The agentic grid: useAgenticGrid + AgenticGridBar

This is the fastest way to let people **ask a grid questions in plain English**. The hook understands the question, filters, sorts, highlights or summarises, and returns the rows for the grid to show. Every answer says which filters it used, so the user can check it.

```
"use client";
import { YuktaiGrid, AgenticGridBar, useAgenticGrid } from "@yuktishaalaa/yuktai";

const columns = [
  { key: "name",   label: "Name",   type: "text"   },
  { key: "salary", label: "Salary", type: "number" },
  { key: "joined", label: "Joined", type: "date"   },
];

export default function StaffPage({ rows }) {
  // The agent: plain-English questions → filters, sort, highlights, summaries
  const grid = useAgenticGrid({ data: rows, columns, rowKey: "id" });

  return (
    <>
      <AgenticGridBar
        grid={grid}
        placeholder="Ask the grid… e.g. salary over 1 lakh in Hyderabad"
        examples={["salary over 1 lakh", "anything unusual?", "average salary by department"]}
      />
      <YuktaiGrid
        data={grid.gridData}
        highlightIds={grid.highlightIds}
        rowKey="id"
        columns={columns.map((c) => ({ ...c, sortable: true }))}
      />
    </>
  );
}
```

Hook options

Option	Purpose
data, columns	Rows and columns (column `type`: text, number or date)
rowKey	ID field (default "id")
referenceDate	"Now" for period questions such as "this month" (default: today)
onOpenRow(id)	Called when the agent opens a row

What the hook returns

Value	Use
gridData	Pass to `<YuktaiGrid data>`; the rows after the question
highlightIds	Pass to `<YuktaiGrid highlightIds>`
ask(question)	Ask a question (async)

Value	Use
reset()	Back to all rows
answer	The plain-English answer to show
kind	"query" | "anomalies" | "summary" | "insight" | "tasks"
table	Rows for a small results table (BI answers)
steps	Steps of a multi-step task, with messages
filters, sort, visibleIds, busy	Current state, for your own UI

Questions that work (tested on 10 sample rows)

Question	kind	Real answer
salary over 1 lakh	query	5 row(s) found. Showing rows where Salary > 1,00,000.
Engineering in Hyderabad	query	1 row(s) found. Showing rows where City is "hyderabad", Department is "engineering".
anything unusual?	anomalies	1 unusual row(s): Salary 5,20,000 is far above the typical range (79,000–1,40,000).
average salary by department	insight	Engineering has the highest average Salary: 2,75,666.67 (across 5 department groups).
show salary over 80k, then sort by salary, then summarize	tasks	Rows: 10 → 7. Filters added: Salary > 80,000. Sorted by Salary low → high.

More phrasings the query parser understands: `salary under 50k`, `top 3 by salary`, `sort by salary descending`, `highest salary first`, `joined after 2025-01-01`, `status overdue` (values are recognised from the data), `name starts with R`.

> **Limits found in testing:** "joined this month" does not filter dates (it returned a total instead); "X or Y", "not active" and "high to low" fall back to a text search; Telugu or Hindi data questions are not understood yet. For those, use the built-in grid assistant with custom rules (section 6).

5.1 Smart grid queries without React

The same parser is available as plain functions, so you can use it on a server, in tests, or with any table component.

```
import { parseAmount, parseGridQuery, executeGridQuery, runGridQuery } from
"@yuktishaalaa/yuktai";

parseAmount("1.5 lakh");   // 150000
parseAmount("2cr");        // 20000000
parseAmount("10k");        // 10000
parseAmount("₹10,000");    // 10000

const parsed = parseGridQuery("salary over 1 lakh", columns, { data: rows });
// parsed.filters     → [{ key: "salary", operator: "greaterThan", value: 100000 }]
// parsed.explanation → "Showing rows where Salary > 1,00,000."
// parsed.notes       → assumptions the parser made (show them to the user)

const result = executeGridQuery(rows, columns, parsed);       // just the rows, no UI
runGridQuery("salary over 1 lakh", { data: rows, columns, rowKey: "id",
  onFiltersChange, onSortChange, onHighlightRows });          // parse + push into the grid
```

5.2 Anomaly finder and summaries

```
import { detectAnomalies, summarizeRows } from "@yuktishaalaa/yuktai";

detectAnomalies(rows, columns, { rowKey: "id" });
// → [{ rowId: "7", column: "salary", kind: "high",
//      reason: "Salary 5,20,000 is far above the typical range (85,000–1,49,250)." }]
// kinds: "high" | "low" | "duplicate"  (interquartile-range rule; tune with factor)

summarizeRows(rows, columns);
// "8 row(s).
//  City: Hyderabad (2), Bengaluru (1), …
//  Salary: total 12,69,000, average 1,58,625, range 68,000–5,20,000.
//  Joined: from 2021-11-11 to 2026-10-07."
```

5.3 BI copilot

```
import { askInsight, groupBy, comparePeriods, trend } from "@yuktishaalaa/yuktai";

askInsight("average salary by department", rows, columns);
// { kind: "group", answer: "Engineering has the highest average Salary: 2,75,666.67 …",
//   table: [{ Department: "Engineering", "avg of Salary": "2,75,666.67", Rows: 3 }, …] }

askInsight("compare salary this month vs last month", rows, columns, { referenceDate: "2026-10-
19" });
// kind "period-change": "Salary dropped 10 (20 → 10, -50.0%) — this month vs the month before."

askInsight("salary trend by month", rows, columns);   // kind "trend"
askInsight("total salary", rows, columns);            // "Total Salary: 12,69,000 across 8
row(s)."

groupBy(rows, "department", "salary", "avg");         // sum | avg | count | min | max
comparePeriods(rows, { dateKey: "joined", metricKey: "salary", period: "month" });
trend(rows, "joined", "salary", "month", "sum");
```

`comparePeriods` compares the same elapsed time by default (for example Monday–Wednesday this week against Monday–Wednesday last week), so a half-finished week does not look like a drop.

5.4 Task runner

```
import { planTasks, runTasks } from "@yuktishaalaa/yuktai";

planTasks("show salary over 1 lakh, then sort by joined, then summarize");
// [{ action: "filter", text: "show salary over 1 lakh" },
//  { action: "sort",   text: "sort by joined" },
//  { action: "summarize", text: "summarize" }]
// actions: filter | highlight | sort | clear | count | summarize | anomalies | open | select

const run = await runTasks("show salary over 1 lakh, then sort by salary, then summarize",
  { data: rows, columns, rowKey: "id", onFiltersChange, onSortChange, onHighlightRows },
  { onStep: (step, i, total) => console.log(i + 1, "/", total, step.message) });
run.summary;  // "Rows: 8 → 4. Filters added: Salary > 1,00,000. Sorted by Salary low → high."
```

6. The built-in grid assistant, agent tools and WebMCP

Every `YuktaiGrid` can also have its own assistant (chat and voice) that drives a set of **grid tools**. The same tools can be handed to browser AI agents through **WebMCP**. This is the best route for Indian-language apps, because you can teach it your own phrases.

```
<YuktaiGrid
  data={rows}
  columns={columns}
  rowKey="id"
  ai                          // built-in assistant: chat + voice
  inputLanguage="te-IN"       // voice input: en-IN, en-US or any of the 22 Indian languages
  webmcp                      // exposes the grid as tools to browser AI agents
  toolName="my_staff_grid"    // tool names: my_staff_grid_search, my_staff_grid_filter, …
  onAgentResult={(r) => setLastAction(r.message)}   // show a plain sentence, never raw JSON
  customRules={[
    {
      name: "high-earners",
      phrases: ["high earners", "top people", "ఎక్కువ జీతం", "ज़्यादा वेतन"],
      description: "Salary over 1 lakh",
      execute: async ({ executeTool }) => {
        const r = await executeTool("filter", { key: "salary", operator: "greaterThan", value:
100000 });
        return r.message;
      },
    },
  ]}
/>
```

6.1 How one request flows

1. The person types or speaks a sentence.
2. **Custom rules are checked first.** If a rule's phrase matches, its `execute` runs and its returned sentence is the answer.
3. Otherwise the built-in parser `parseGridIntent` (or your own `parseIntent`) picks a tool and its input.
4. Required inputs are checked against the tool's schema, then the tool runs.
5. Every tool returns `{ success, message, data?, error?: { code } }`. The message is in English or Telugu.
6. The step is saved in history (20 by default) and `onAgentResult` is called.

6.2 Built-in phrases (tested)

Person says	Tool	Notes
how many · count · ఎన్ని	count	Counts the rows shown
search X · find X · X వెతుకు	search	
open X · X తెరువు	search → open	Opens the first match; needs `onRowClick` / `onOpenRow`
sort by title · sort by salary desc · శీర్షిక క్రమంలో	sort	Column matched by key or label
clear filters · clear sort	clear_filters · clear_sort	
anything else (including Hindi today)	search	The whole sentence becomes the search

Person says	Tool	Notes
		text

Filter, highlight, select, columns and get_row are not phrase-parsed. Reach them with a custom rule (as above) or `executeTool`.

6.3 The 11 grid tools

Tool	Telugu label	Required input	Created only when
search	వెతుకు	query	always
count	లెక్క	—	always
columns	కాలమ్‌లు	—	always
get_row	వరుస చూపు	id	always
highlight	హైలైట్	ids	`onHighlightRows` is given
select	ఎంచుకో	id	`onSelectRow` is given
open	తెరువు	id	`onOpenRow` is given
filter	ఫిల్టర్	key, operator, value	`onFiltersChange` is given
clear_filters	ఫిల్టర్లు తీసేయి	—	`onFiltersChange` is given
sort	క్రమం	key (direction optional)	`onSortChange` is given
clear_sort	క్రమం తీసేయి	—	`onSortChange` is given

- A tool is only created when the grid can really perform it. Tested: with no callbacks you get only search, count, columns and get_row.
- Filter operators: `contains`, `equals`, `startsWith`, `endsWith`, `greaterThan`, `lessThan`, `between` (value `[min, max]`).
- Tool messages come in English (`locale: "en"`) or Telugu (`locale: "te"`). Tested: filter → "ఫిల్టర్ వేశాను: 1 వరుసలు సరిపోయాయి."

Error code	Meaning
NOT_FOUND	Row or column does not exist (tested: get_row "9" → ""9" వరుస దొరకలేదు.")
NOT_SUPPORTED	The grid has no callback for this action
INVALID_INPUT	A required input is missing or wrong
INTERNAL	Failure inside a tool; tools never throw
NOT_UNDERSTOOD	Agent only: nothing matched the sentence

6.4 Custom rules for your own words (any language)

Field	Purpose
name	Rule id for debugging
phrases	Exact phrases that activate it, in any script
description	Optional documentation

Field	Purpose
execute(context)	Returns the answer sentence. `context` has `input`, `data`, `columns` and `executeTool(name, input)`

- Add the phrase in every language your users speak: English, Telugu, Hindi…
- Return one plain sentence; change the grid only through `executeTool`, so the assistant and WebMCP stay in sync.
- Always add a `help` rule that lists what people can ask.

6.5 Your own agent: useYuktaiGridAgent

```
import { createGridTools, useYuktaiGridAgent, YuktaiGridAI } from "@yuktishaalaa/yuktai";

const tools = createGridTools(
  { data: rows, columns, rowKey: "id", locale: "te",
    onOpenRow, onFiltersChange: setFilters, onSortChange: setSort },
  { name: "my_staff_grid" },
);
const agent = useYuktaiGridAgent({ tools, data: rows, columns, rowKey: "id", locale: "te",
  customRules: rules, historyLimit: 20, onResult: (r) => setLastAction(r.message) });

await agent.ask("ఎన్ని");                                  // sentence → tool
await agent.executeTool("my_staff_grid_sort", { key: "salary", direction: "desc" });
// also: loading, lastResult, lastError, history, clearHistory(), tools

<YuktaiGridAI data={rows} columns={columns} language="te-IN" agent={agent} embedded />
```

6.6 WebMCP: the same tools for browser AI agents

```
<YuktaiGrid data={rows} columns={columns} ai webmcp toolName="my_staff_grid"
  toolDescriptions={{ search: "Search staff by name, city or department." }}
  onWebMCPStatusChange={setStatus} />

// or without a visible grid
<YuktaiGridWebMCP data={rows} columns={columns} name="my_staff_grid"
onStatusChange={setStatus} />
```

Status	Meaning
unsupported	The browser has no `document.modelContext` (normal in most browsers today)
registering	Registering tools
ready	All tools registered (`registered` lists them)
partial	Some failed; see `errors[]`
error	Registration failed

Write `toolDescriptions` in English, because they are read by AI agents and never shown in the UI. Keep labels and messages in your users' language.

7. Page agents

These agents look at the current page (or any element you pass as `root`) and report in plain English. They are useful in development, in QA, and as user-facing helpers.

7.1 QA tester — runQaAudit

```
import { runQaAudit, auditToMarkdown, auditToJSON } from "@yuktishaalaa/yuktai";

const report = runQaAudit();          // 15 WCAG checks on the current page
report.score;                         // 0–100
report.summary;                       // "9 of 15 checks passed. 6 issue(s) found, 3 critical.
Score 58/100."
report.issues[0].fix;                 // what to change, in plain English
const md = auditToMarkdown(report);   // paste into a ticket or PR
const json = auditToJSON(report);     // no DOM references: safe to store or send

// Options: root, skip: ["color-contrast"], maxContrastChecks (600), minTargetSize (24 px)
```

Each issue has `rule`, `severity`, `wcag`, `message`, `why`, `fix`, `selector`, `snippet` and `autoFixable`. The 15 rules (from `AUDIT_RULES`):

Rule id	Check	WCAG	Severity
html-lang	Page language is set	3.1.1	serious
page-title	Page has a title	2.4.2	serious
image-alt	Images have alternative text	1.1.1	critical
heading-order	Headings follow a logical order	1.3.1	moderate
single-h1	Page has one main heading	best practice	minor
landmark-main	Main content landmark exists	1.3.1 / 2.4.1	moderate
form-label	Form fields have labels	1.3.1 / 4.1.2	critical
button-name	Buttons have names	4.1.2	critical
link-name	Links have names	2.4.4	serious
color-contrast	Text has enough contrast	1.4.3	serious
positive-tabindex	No positive tabindex	2.4.3	moderate
keyboard-access	Clickable things work with a keyboard	2.1.1	critical
duplicate-id	IDs are unique	best practice	minor
form-error	Errors are linked to their fields	3.3.1	serious
target-size	Touch targets are large enough	2.5.8	moderate

7.2 Accessibility copilot — createA11yCopilot

```
import { createA11yCopilot } from "@yuktishaalaa/yuktai";

const copilot = createA11yCopilot({ onUpdate: (s) => setState(s), autoApply: false, debounceMs:
600 });
copilot.start();                      // watch the page and rescan after DOM changes
const report = copilot.scan();
copilot.explain(report.issues[0].id);  // "The <html> element has no lang attribute.
```

```
                                        //  Why it matters: Screen readers pick their voice …
Telugu or Hindi
                                        //  text may be read with English rules. …"
copilot.applyFix("html-lang-1");        // { applied: true, message: 'Set lang="en-US" on <html>.'
}
copilot.applyAllFixes();
copilot.undoAll();                      // every automatic change can be undone
copilot.stop();
```

Use `defaultLang` (for example `"te"` or `"hi"`) so the language fix sets the right language for your site instead of the browser default.

7.3 Design review — reviewDesign

```
import { reviewDesign } from "@yuktishaalaa/yuktai";

const d = reviewDesign({ minFontSize: 16, maxLineLength: 90 });  // 16 px recommended for seniors
d.score;      // 0–100
d.summary;    // "Design score 98/100 with 1 finding(s). Weakest area: hierarchy."
d.categories; // hierarchy, readability, interaction, clutter, consistency → { score, findings }
d.findings;   // [{ category, severity, message, suggestion, selector }]
```

7.4 Form review — reviewForms

```
import { reviewForms, reviewForm, readFormFields } from "@yuktishaalaa/yuktai";

reviewForms({ seniorFriendly: true, maxFieldsPerStep: 8 });
// [{ score: 73, summary: '"f" has 2 field(s): 1 problem(s) and 3 suggestion(s). Score 73/100.',
//    findings: ['A text field ("email") has no label.', '"email" has no autocomplete hint.',
…] }]

reviewForm(document.querySelector("form")!);   // one form
readFormFields(formElement);                   // label, type, name, required, hasVisibleLabel
```

Senior-friendly checks cover font size, number of fields per step and vague button text, which matters for apps used by older citizens.

7.5 Page understanding — askAboutPage

```
import { askAboutPage, analyzePage, detectPageIntent } from "@yuktishaalaa/yuktai";

const a = await askAboutPage("Which fields have no label?");
a.intent;       // "missing-labels"
a.answer;       // '1 of 2 field(s) have no label: text ("email").'
a.suggestions;  // ["Add a visible <label for=…> to each field.", …]
a.targets;      // selectors the UI can highlight

analyzePage();  // title, language, headings, landmarks, counts, unlabeled fields, images without
alt
```

Intent	Example question	Tested answer
purpose	What is this page about?	"Staff directory" lets you fill in 1 form(s).
missing-labels	Which fields have no label?	1 of 2 field(s) have no label: text ("email").
links / counts	How many links are there?	2 links, 1 buttons, 2 form fields in 1 form(s), 1 images, 0 tables.

Intent	Example question	Tested answer
structure	Show the page structure	2 heading(s) … Missing landmarks: header, navigation, search, footer.
images	Which images have no alt text?	1 of 2 image(s) have no alt text.
forms	Tell me about the forms	There are no forms on this page. (on a page without forms)

For "purpose" questions you can pass `summarize: (pageText) => myModel(pageText)` to get a model-written summary. Answers are in English.

7.6 Page + data RAG — createContextIndex

```
import { createContextIndex, tokenize } from "@yuktishaalaa/yuktai";

const index = createContextIndex({
  includePage: true,                      // index the visible page text
  rows, columns, rowKey: "id",            // index grid rows (including other pages)
  documents: [{ id: "faq1", text: "సెలవుల కోసం HR విభాగాన్ని సంప్రదించండి." }],
});

index.search("Hyderabad Engineering", 3); // BM25 keyword hits with score and source
index.search("సెలవుల");                   // Telugu works: finds faq1
const a = await index.ask("who works in Pune?");          // extractive: "Row 7: City: Pune"
await index.ask(q, { generate: (q, ctx) => myModel(q, ctx) }); // optional model-written answer
index.rebuild();                          // after the page or data changes

tokenize("తెల్లగ్ పద్యాలు");                 // keeps Telugu/Devanagari combining marks together
```

7.7 Voice commands

```
import { parseVoiceA11yCommand, createVoiceA11yController } from "@yuktishaalaa/yuktai";

parseVoiceA11yCommand("make text bigger", { fontSizeMultiplier: 1 });
// { type: "setting", patch: { fontSizeMultiplier: 1.1 }, say: "Text size 110%." }
parseVoiceA11yCommand("next row");   // { type: "grid", action: "next-row", say: "Next row." }

const voice = createVoiceA11yController({ lang: "te-IN", speak: true });
voice.start();                        // listens with the browser's speech recognition
await voice.handle("పెద్ద అక్షరాలు");   // typed commands work too (handy for tests)
```

Command	English phrases (tested)	Telugu phrases (tested)
Bigger / smaller text	make text bigger	పెద్ద అక్షరాలు · చిన్న అక్షరాలు
High contrast	high contrast on	హై కాంట్రాస్ట్
Dark mode	dark mode · dark mode off	డార్క్ మోడ్
Reduce motion / dyslexia font / read aloud	reduce motion · dyslexia font · read aloud	—
Grid rows	next row · previous row · first row · last row · open row	తర్వాతి వరుస · మొదటి వరుస
Reset	reset	రీసెట్

Text size moves in steps of 10% between 80% and 130%. When nothing matches, the parser returns `null`, so you can pass the sentence on to another handler.

8. All page tools in one panel: AgenticPanel

```
"use client";
import { AgenticPanel } from "@yuktishaalaa/yuktai";

export default function PageTools() {
  return (
    <AgenticPanel
      defaultTab="qa"              // copilot | qa | forms | design | ask | voice
      voiceLang="te-IN"            // speech recognition language for the Voice tab (default "en-
IN")
      generate={async (q, ctx) => myModel(q, ctx)}   // optional, for the Ask tab
    />
  );
}
```

Six tabs: **Copilot**, **QA report**, **Forms**, **Design**, **Ask page** and **Voice**. The panel marks itself with `data-yuktai-panel`, so the agents never audit the panel's own UI.

9. Core modules

9.1 Accessibility engine

- WCAG 2.2 auto-fix: ARIA, roles, tabindex, scope, autocomplete, alt text; watches new DOM elements.
- Speak on focus and voice control with the browser's speech APIs; visual alerts for deaf and hard-of-hearing users.
- High contrast, dark mode, reduce motion, large targets (44×44), colour-blind modes, Atkinson Hyperlegible dyslexia font, font scaling 80–130%, skip links, focus trap, saved preferences.

```
import { wcagPlugin, type A11yConfig } from "@yuktishaalaa/yuktai";

const report = wcagPlugin.applyFixes({ enabled: true, autoFix: true, highContrast: false,
  colorBlindMode: "none", fontSizeMultiplier: 1 } as A11yConfig);
report.fixed;  // number of fixes
report.score;  // 0–100
```

9.2 In-tab RAG (Ask this page)

Desktop Chrome with Gemini Nano answers from the page text directly. Mobile and other browsers use Transformers.js with a quantised `flan-t5-small` model (about 30 MB once, cached in IndexedDB, then fully offline).

9.3 Autonomous agent

Type a goal in plain English; the agent reads the page with 9 DOM strategies and 11 form-label strategies, plans numbered steps, highlights the target field and scrolls to it. It works on static HTML, React, WordPress and older government portals, with a rule-based fallback.

9.4 Vibe Coder

Describe a site and download a Next.js 16 project as a ZIP (TypeScript, Tailwind + CSS Modules, mobile responsive). It is template engineering: no AI writes the code.

9.5 Gemini Nano setup (desktop Chrome, optional)

1. Open `chrome://flags` and enable the Prompt API for Gemini Nano, Summarization, Writer, Rewriter and Translation APIs.
2. Relaunch Chrome.
3. Open `chrome://components`, find **Optimization Guide On Device Model** and click **Check for update**.
4. Reload your page. Mobile and other browsers fall back to Transformers.js automatically.

9.6 Icons

```
import { SearchIcon, SortUpIcon, SortDownIcon, ChevronLeftIcon, ChevronRightIcon, CheckIcon,
CloseIcon } from "@yuktishaalaa/yuktai";
<CheckIcon size={20} color="#047857" label="Task complete" />   // label adds ARIA; omit for
decorative
```

10. Large datasets (100K+ rows)

Client-side pagination is right for a few thousand rows. For more, page on the server and turn off the grid's own search and pagination. The browser then only holds one page.

```
// app/api/staff/route.ts — returns one page at a time
export async function GET(req: NextRequest) {
  const p = req.nextUrl.searchParams;
  const page = Math.max(1, parseInt(p.get("page") || "1"));
  const pageSize = Math.max(1, parseInt(p.get("pageSize") || "500"));
  const { rows, total } = await db.staff.page(page, pageSize, p.get("search") ?? "");
  return NextResponse.json({ rows, pagination: { page, pageSize, totalRows: total, totalPages:
Math.ceil(total / pageSize) } });
}

// page
<YuktaiGrid data={apiResp?.rows ?? []} columns={columns}
  search={false} pagination={false} loading={!apiResp} />
```

Note: the agentic functions work on the rows they are given. With server paging, run them on the current page, or call them on the server with the full data.

11. Plug in your own model (optional)

Everything works offline with rules and keyword search. To get model-written answers, pass a hook. Only what you pass to the hook leaves the device.

Where	Hook
AgenticPanel (Ask tab)	`generate(question, context)`
createContextIndex().ask	`{ generate(question, context) }`
askAboutPage (purpose questions)	`{ summarize(pageText) }`

12. Honest limits

- Questions are understood with rules, not a language model, so unusual phrasing may need rewording. The parser returns `notes` explaining its assumptions; show them to users.
- Smart grid queries, page answers, BI answers and reports are in English today. Telugu and Hindi data questions are not parsed yet; use custom rules for them.
- Voice accessibility commands understand English and Telugu, not yet Hindi or other languages.
- QA and design checks are automatic heuristics. They catch common problems but do not replace testing with real users and assistive technology.
- WebMCP registration only works in browsers that provide `document.modelContext`; always show the real status.

13. Compatibility, principles and links

Area	Supported
Next.js	13, 14, 15, 16
React	17, 18, 19 (19 needs `--legacy-peer-deps`)
Node.js	18+
Browsers	Chrome 90+, Firefox 90+, Safari 15+, Edge 90+, Samsung Internet 14+
Languages	English; Telugu UI and phrases; voice input in 22 Indian languages; Indian number format

Design principles

- Zero API keys, zero cost, zero telemetry; everything runs in the browser.
- SSR safe; never reads its own UI (`data-yuktai-panel`); every automatic fix can be undone.
- Honest limits: client-side pagination for thousands of rows, server paging beyond that.

Developer checklist

- Set `lang` on `<html>` and `locale` / `inputLanguage` on the grid for your users' language.
- Give the grid a stable `rowKey`; pass only the callbacks your page supports.
- Add custom rules with phrases in every language you serve, plus a `help` rule.
- Show agent answers and WebMCP status as plain sentences.
- Run `runQaAudit()` and `reviewForms()` before each release; keep the Markdown report with the pull request.

Links

- npm: npmjs.com/package/@yuktishaalaa/yuktai
- GitHub: github.com/sandeepmiriyala03/yuktai
- Licence: GPL-3.0 © Sandeep Miriyala, Yuktishaalaa AI Lab

Verification notes

Checked on @yuktishaalaa/yuktai 5.2.0 (latest on npm at the time of writing). Functions were run in Node with a browser-like DOM (happy-dom); `useAgenticGrid` was rendered with React 19. Sample answers in this guide are copied from those runs. Results on a real page will differ with its content.